

Block Diffusion 에서의 D-CBG (Discrete Classifier-Based Guidance)

Schiff et al. 분류기 가이드의 블록 확산 정식화

Block-Diffusion-OD-Planning (v2 backbones) — 자매 문서: BD_BLOCK_GUIDANCE_MATH.pdf

2026-07-21

Abstract

본 문서는 Schiff et al. 의 **D-CBG**(discrete classifier-based guidance)를 block diffusion 상황에서 수식으로 정리한다. IW guidance(BD_BLOCK_GUIDANCE_MATH.pdf)와의 핵심 차이는 두 가지다: (i) 분류기가 **clean** 부분 경로가 아니라 시각 t 의 노이즈 상태 x_t 를 조건으로 받으며 시간 t 로 조건화된다는 점, (ii) guidance가 후보 재가중(importance sampling)이 아니라 **역과정 posterior**를 **위치별로 직접 tilting** 한다는 점이다. 두 계산 변형—*exact enumeration*과 *first-order (Taylor)*—를 두 커널(mask, graph)에 대해 각각 수식화하고 실제 구현(dcbg_plugin.py)의 라인을 병기한다. plug-in 이식(그들 코드 로직 그대로)이며, 분류기 아키텍처·학습 데이터는 IW disc와 통제되어 남는 차이가 방법 내재적임을 명확히 한다.

1 표기와 D-CBG 목표

자매 문서와 동일한 block-diffusion 세팅을 쓴다: canvas를 크기 B 블록으로 나누고, 블록 b 를 디노이징할 때 이전 블록 $x^{<b}$ 는 확정(commit), 현재 블록 x_t^b 는 시각 t 의 latent이다. 조건 변수 $y \in \{\text{normal}, \text{exceptional}\}$ 에 대해 D-CBG는 조건부 역과정에서 샘플하려 한다:

$$p(x_{t-1}^b \mid x_t^b, x^{<b}, y) \propto p_\theta(x_{t-1}^b \mid x_t^b, x^{<b}) \cdot p(y \mid x_{t-1}^b, x_t^b, x^{<b}). \quad (1)$$

$p(y \mid \cdot)$ 를 직접 모르므로, 시각 t 로 조건화된 노이즈-상태 분류기 $p_\phi(y \mid x_t, t)$ 로 근사한다. 이것이 IW와의 첫 번째 구조적 차이이다(IW는 clean x_0 의 밀도비를 쓴다).

2 노이즈-상태 분류기 $p_\phi(y \mid x_t, t)$

분류기는 canvas 형태 노이즈 시퀀스를 입력받는다(vocab = 정점 $\cup \{\text{END}, \text{PAD}, \text{MASK}\}$), 시간 임베딩으로 t 조건화, transformer + masked mean pooling, 그리고 log-odds를 출력한다(dcbg_plugin.py:70--91):

$$z_\phi(x_t, t), \quad [\log p_\phi(y=0 \mid x_t, t), \log p_\phi(y=1 \mid x_t, t)] = [\log \sigma(-z_\phi), \log \sigma(z_\phi)]. \quad (2)$$

생성-정합 corruption 으로 학습. 블록 확산의 생성 시점 상태를 그대로 재현해 학습한다:

- **mask 커널**(corrupt_mask, dcbg_plugin.py:107): j 개의 확정 *clean* 블록 + 현재 블록을 rate t 로 마스킹(first-hitting marginal), 미래 블록은 부재. 즉 분류기도 “확정 prefix $x^{<b}$ + 노이즈 현재 블록”을 본다.
- **graph 커널**(corrupt_graph, dcbg_plugin.py:132): 현재 블록 전체를 $q(x_t \mid x_0) = \text{Cat}(x_0 \bar{Q}_t)$ 로 정수 t 에서 노이징, prefix는 clean.

Remark 1 (IW disc 와의 통제·차이). `DCBGClassifierAdj(dcbg_plugin.py:393)`는 IW 판별자와 동일한 GraphSAGE + 전이 피쳐[edge-exists, is-transition, deg-ratio] + bounded logit $\lambda \tanh(z/\lambda)$ 을 쓴다(:408--443). 학습 데이터·negative pool·백본도 통제한다. 유일하게 남는 차이는 입력이 노이즈 $x_t + t$ 라는 점이며, 이는 D-CBG 수식이 내재적으로 요구하는 것이다.

Remark 2 (블록 적합성: mask 는 맞고 graph 는 non-block 원본이 남음). block diffusion 에 맞는 노이즈 상태는 이전 블록 clean 고정 + 현재 블록만 노이즈 + 미래 블록 부재이다.

- **mask:** `corrupt_mask(:107)`가 j 개 확정 블록을 clean 유지, 현재 블록만 rate t 마스킹, 미래 truncate 하여 이 구조를 정확히 구현한다. 추론 (`plan_dcbg_mask`)도 full canvas(prefix+ 현재 블록)를 넣어 학습과 일치한다.
- **graph:** `corrupt_graph(:145)`는 `row[2:]=noised[2:]` 로 경로 전체를 단일 t 로 노이즈한다(dst, ori 만 clean) — 이는 블록 구조가 아니라 *non-block MDLM/uniform* 원본 corruption 이다. 게다가 추론 (`plan_dcbg_graph:320`)은 `seq[:,-block:]` 로 현재 블록만 넣어 prefix 를 무시하고 학습/추론 입력이 불일치한다. 즉 graph D-CBG 는 블록 확산에 미적응 상태이며, 이것이 §5 의 graph 무력성에 (Taylor 근사 무력과 함께) 겹친다.

Remark 3 (Block-size 독립성: IW 는 가능, D-CBG 는 불가). IW disc 는 clean x_0 공간의 비율을 다루고 입력이 *clean* 부분 경로라 (`bd_disc.py:119 forward` 에 block 인자 없음) **block size** 에 무관하다 — 실제로 자매 문서 실험 §4.1b 에서 단일 disc 하나로 blk 4/8/16/32/64 를 모두 평가했다. 반면 D-CBG 는 $p_\phi(y | x_t, t)$ 를 다루는데 x_t 의 구조(어디가 확정 clean 블록/현재 노이즈 블록/미래 부재인지)가 **block size** 로 정의되므로, `corrupt_*` 가 block 을, `train_dcbg_classifier.py` 가 -blk 를 요구하며 블록마다 별도 `classifier(DCBGclf_*.blk{B})` 가 필요하다. 이는 구현 편의가 아니라 정식화의 구조적 차이이며, 블록 확산에서 x_0 -space(IW) 정식화가 갖는 실질적 이점이다.

3 위치별 posterior tilting

식 (1) 를 이산 factorized 역과정에 적용하면, 위치 ℓ 의 다음 값 k 에 대해

$$p(x_{t-1}^b{}^\ell = k | x_t, x^{<b}, y) \propto p_\theta(x_{t-1}^b{}^\ell = k | x_t, x^{<b}) \cdot p_\phi(y | x_t^{\ell \rightarrow k}, t)^\gamma, \quad (3)$$

즉 로그 공간에서 덧셈 **tilt**: $\log p_\theta(\cdot = k) + \gamma \log p_\phi(y | x_t^{\ell \rightarrow k}, t)$ 후 softmax. 여기서 $x_t^{\ell \rightarrow k}$ 는 현재 노이즈 상태 x_t 의 위치 ℓ 만 k 로 치환한 것으로, $p(y | x_{t-1}^b)$ 의 tractable 대리(surrogate)이다. γ 는 guidance 세기(우리 실험 $\gamma = 4$; $\gamma \leq 2$ 는 무력, 보고서 §5). 이것이 IW 와의 두 번째 구조적 차이다: IW 는 joint 후보 x_0 를 self-normalized 재가중하는 반면, D-CBG 는 위치별 *reverse categorical* 을 직접 기울인다.

4 계산 변형 1 — Exact enumeration (mask)

mask first-hitting 은 매 reveal 에서 한 위치 $\ell(=idx)$ 만 소비하므로, 식 (3) 의 열거를 그 위치로 제한할 수 있다. 그 위치를 vocab 의 모든 값 k 로 치환한 $x_t^{\ell \rightarrow k}$ 를 만들어 분류기에 통과시킨다:

$$\text{guided}(k) = \log p_\theta(x_{t-1}^b{}^\ell = k | x_t, x^{<b}) + \gamma \log p_\phi(y | x_t^{\ell \rightarrow k}, t), \quad x_{t-1}^b{}^\ell \sim \text{softmax}_k \text{guided}(k). \quad (4)$$

구현: `xt_jumps = x_t^{\ell \rightarrow k}` 를 (b, V, s) 로 만들고(`dcbg_plugin.py:222--224`), `clf.get_log_probs` 를 마이 크로배치로 평가(:227--231), `guided = log_p[idx] + gamma*clf_lp` 후 샘플(:233--235). **비용:** reveal 당 분류기 호출 = $|V| \approx 1,393$ (reveal 당 forward V 회).